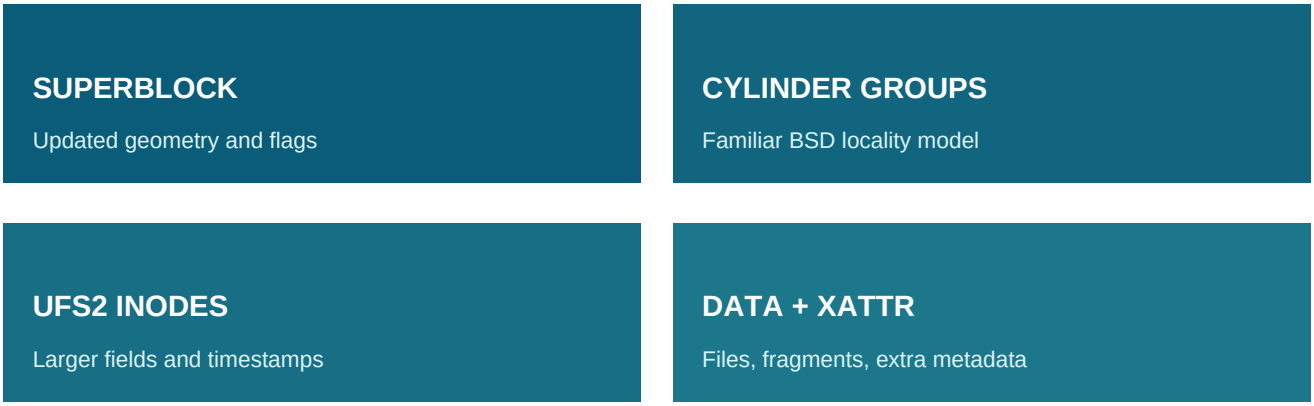
UFS2 - BSD Fast File System successor

HISTORY / ORIGINAL DESIGN / DISK FORMAT

The 2003-era BSD on-disk evolution for multi-terabyte storage, larger metadata and improved timestamp and inode representations.

This is a technical reference to the upstream filesystem's historical design, not an installer manual. The format's on-media semantics are distinct from the independent InfiltratorOS module, its APIs and any Linux kernel adapter.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

UFS2 - BSD Fast File System successor

HISTORICAL CHRONOLOGY

UFS2 was developed for FreeBSD during the transition to multi-terabyte storage. Marshall Kirk McKusick's BSDCon 2003 paper describes the reasons for a new on-disk format rather than indefinite enlargement of UFS1. It retained the BSD cylinder-group and Fast File System philosophy while changing structures that were becoming too small or too restrictive.

The design introduced wider block pointers, higher-resolution timestamps, extended attributes and a larger boot block area, with modifications to soft updates. It built on decades of BSD storage design rather than attempting to be a ZFS-style pooled copy-on-write filesystem. UFS1 remained an important compatibility format, but UFS2 should be identified separately.

UFS2 - BSD Fast File System successor

ORIGINAL ON-DISK ARCHITECTURE

UFS2 has a superblock, cylinder groups and inodes analogous in purpose to UFS1, but the on-disk inode structure is different. UFS2 stores larger block addresses and extended metadata, enabling increased file and volume capacities. A valid reader must parse the format version and its exact inode fields instead of casting a UFS1 disk inode to a UFS2 structure.

Each cylinder group maintains block/inode allocation metadata and locality information. Data blocks can be divided into fragments. Extended attributes may be associated with separately allocated blocks, and their allocation/deallocation dependencies have implications for consistency. Updated timestamp structures and inode metadata were explicit design goals.

UFS2 - BSD Fast File System successor

ALLOCATION AND DIRECTORIES

Cylinder-group placement strategies aim to keep related data near its inode and disperse directories sufficiently for balance. Bitmaps and summary counts permit efficient free-space selection. Direct and indirect block pointers map the file into allocated storage. As with UFS1, the exact algorithm used by the operating system is a policy layered on the persistent format.

Directory entries carry inode references, record lengths and names. Hard links, ownership, permissions and other Unix behaviour require consistent inode metadata. UFS2’s metadata structures support extended attributes, and directory or inode corruption must be handled with format-specific bounds and revision awareness.

UFS2 - BSD Fast File System successor

INTEGRITY AND RECOVERY

UFS2 participates in BSD’s soft-updates model, which tracks write dependencies and enforces an order that avoids certain inconsistent allocation states without requiring every metadata write to be synchronous. Soft updates do not journal all file data. Some deployments may use a journaled variant or soft-updates journalling, but the presence and policy must be discovered rather than assumed.

The 2003 engineering work addressed extended attribute dependencies, cleaner free-space accounting and scalable recovery operations. Offline fsck remains important when metadata is damaged. A modern repair tool checks group data, ownership maps, extended metadata and directory connectivity instead of treating a consistent superblock as a complete audit.

UFS2 - BSD Fast File System successor

DESIGN RATIONALE AND LIMITATIONS

UFS2 deliberately preserves familiar BSD Fast File System allocation strategies while expanding the representable disk geometry and inode capabilities. It balances proven locality-based design and backward-looking operational experience with the demands of much larger storage systems.

It is not interchangeable with UFS1. Neither should be described as having inherent ZFS-style snapshots, checksums or transactional copy-on-write. Filesystem features, write durability and repair behaviour depend on precise FreeBSD implementations and mounted options.

UFS2 - BSD Fast File System successor

INFILTRATOROS INTEGRATION BOUNDARY

UFS2.so must be tested against independently formatted UFS2 media. Shared source with UFS1 is an implementation convenience, not evidence that both disks can be parsed identically. Validate superblock identity, inode format and endianness before claiming read or write support.

Our release package, optional ‘.ko‘ and filesystem documentation are independent deliverables. The original BSDCon paper remains the primary rationale for UFS2’s on-disk architectural changes.

UFS2 - BSD Fast File System successor

SOURCES AND PRIMARY REFERENCES

The documents below are the historical and engineering reference points used for this guide. Where an on-media field or feature flag is ambiguous, consult the original format documentation and its version-specific implementation notes before interpreting or changing media.

1. McKusick, Enhancements to the Fast Filesystem to Support Multi-Terabyte Storage Systems (2003)

https://static.usenix.org/events/bsdcon03/tech/full_papers/mckusick/mckusick_html/index.html

2. Original Berkeley Fast File System design paper

https://web.eecs.umich.edu/~prabal/teaching/eecs582-w11/readings/mckusick84ffs.pdf

3. Soft Updates technical paper (1999)

https://www.usenix.org/conference/1999-usenix-annual-technical-conference/soft-updates-technique-eliminating-most

The InfiltratorOS shared-object module, optional kernel adapter, native builder and release-specific tests are a separate implementation. A successful document build or package installation does not qualify physical writes, recovery, kernel mounting or boot.

UFS2 / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Identify the UFS2 superblock and its versioned filesystem geometry. The format preserves BSD cylinder groups while changing core inode and address structures.

2. Use the correct UFS2 inode structure with wider block addresses, updated timestamps and separately managed extended-attribute metadata.

3. Map an inode number through the cylinder-group geometry and inode-table layout, not by reusing a UFS1 on-disk inode size.

4. Decode direct and multi-level indirect references using the width defined by UFS2 and validate fragment versus whole-block allocation.

5. Read cylinder-group free maps, fragment summaries and inode-use metadata. Their interpretation must remain consistent with the larger address space.

6. Check directory records and hard-link counts without assuming that larger UFS2 disk addresses change the ordinary Unix pathname model.

7. Interpret extended attributes and any separate attribute blocks as part of persistent inode state; they can create additional allocation dependencies.

8. Distinguish format design changes from soft-updates and other kernel implementation techniques that can operate around the same underlying media layout.

UFS2 / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Never truncate UFS2 block numbers or inode references to UFS1 widths. Silent narrowing can produce convincing but unrelated physical addresses.

2. Verify superblock parameters and cylinder-group descriptors with full-width arithmetic before following any data pointer.

3. Check fragment allocation and indirect block traversal for overlaps, cycles and inconsistent inode block counts.

4. Include extended-attribute storage in ownership analysis, not just the visible contents of the data fork.

5. Soft updates address ordering constraints but do not guarantee complete user-data journaling. Treat recovery according to the actual deployed update policy.

6. Corroborate alternate superblocks and group metadata before repairing a primary header; backups need not describe the latest complete state.

7. Do not assume that UFS1 disk inode offsets are a compatible subset of UFS2. Readers and writers must select the format-specific decoder.

8. Reference the original BSD FFS and UFS2 engineering papers for the exact revision’s limits and design rationale rather than extrapolating from modern kernel flags.